AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Computer Engineering and Software Systems
Computer and Artificial Intelligence Engineering

MicroGrad

Scalar Autograd Engine

Assignment 1 — Technical Documentation

CSE485–CSE488: Deep Learning

Student name: Omar Osama Ibrahim

Student ID: 2300090

Submission date: 3/10/2026

Based on the implemented scalar engine, neural-network API, and executed XOR demonstration notebook.

Contents

1 Objective and Scope **2**

2 Scalar Computational Graph **2**
2.1 The `Value` abstraction . . . 2
2.2 Local derivative rules . . . 2
2.3 Worked gradient example . . . 2

3 Topological Ordering and Backpropagation **2**

4 Neural Network API and Design **3**
4.1 Neuron and layer . . . 3
4.2 MLP architecture . . . 3

5 Experiment: Learning XOR **3**
5.1 Dataset and loss . . . 3
5.2 Training procedure . . . 3

6 Results and Discussion **4**

7 Verification and Limitations **4**

8 Conclusion **5**

1 Objective and Scope

This assignment implements a small scalar automatic differentiation system inspired by MicroGrad. The objective is to construct a computational graph during a forward pass, calculate local derivatives, and apply reverse-mode automatic differentiation through the chain rule. The implementation avoids using PyTorch as the differentiation engine. A small neural-network API (`Neuron`, `Layer`, and `MLP`) is then built on top of the scalar `Value` class.

2 Scalar Computational Graph

2.1 The `Value` abstraction

Each `Value` object stores a scalar `data` value, an accumulated gradient `grad`, its parent nodes `_prev`, the creating operation `_op`, and a local backward closure `_backward`. Mathematical operators create new `Value` nodes while recording dependencies. Numeric scalar operands are wrapped as `Value` objects where needed.

2.2 Local derivative rules

Let L be the final scalar output and $g_y = \partial L/\partial y$ the upstream gradient at node y. Each operation contributes to its inputs by the chain rule:

$$\begin{aligned} y = a + b: &\quad g_a \mathrel{+}= g_y, \quad g_b \mathrel{+}= g_y \\ y = ab: &\quad g_a \mathrel{+}= b\, g_y, \quad g_b \mathrel{+}= a\, g_y \\ y = a^n: &\quad g_a \mathrel{+}= n a^{n-1} g_y \\ y = \tanh(a): &\quad g_a \mathrel{+}= (1 - \tanh^2(a)) g_y \\ y = e^a: &\quad g_a \mathrel{+}= e^a g_y. \end{aligned}$$

Subtraction is expressed as addition of a negated value; division uses multiplication by a reciprocal power. Accumulation with `+=` is essential because a variable may influence the output through multiple graph paths.

2.3 Worked gradient example

For $a = 2$, $b = 3$, and $L = ab + a$, the forward value is $L = 8$. Differentiating yields

$$\frac{\partial L}{\partial a} = b + 1 = 4, \qquad \frac{\partial L}{\partial b} = a = 2.$$

The second contribution to $\partial L/\partial a$ demonstrates why overwriting a gradient would be incorrect.

3 Topological Ordering and Backpropagation

A depth-first traversal begins at the output node, maintains a `visited` set, recursively explores `_prev`, and appends each node *after* visiting its dependencies. This post-order list is a valid topological ordering. Backpropagation seeds the output gradient with 1, since $\partial L/\partial L = 1$, then executes local backward closures in reverse topological order.

1. Traverse each reachable node only once while constructing the ordering.
2. Append nodes after their dependencies, not before them.
3. Set the final scalar output gradient to 1.0.
4. Iterate over the reversed ordering, accumulating each local derivative contribution.

The visited set prevents repeated traversal of shared graph nodes. It does *not* replace gradient accumulation: for $L = a + a$, the correct derivative is $\partial L/\partial a = 2$.

4 Neural Network API and Design

4.1 Neuron and layer

A neuron with n inputs computes

$$z = \sum_{i=1}^{n} w_i x_i + b, \qquad h = \tanh(z).$$

Weights and bias are initialized independently from a uniform distribution on $[-1, 1]$ and stored as trainable `Value` objects. A `Layer` holds multiple neurons, evaluates them on the same input, and returns a scalar for a one-neuron layer or a list otherwise. Each class exposes its trainable parameters and a `zero_grad()` method.

4.2 MLP architecture

The demonstration uses `MLP(2, [4, 4, 1])`: two inputs, two hidden layers with four neurons each, and a one-neuron output layer. The parameter count is

$$(2+1)\cdot 4 + (4+1)\cdot 4 + (4+1)\cdot 1 = 12 + 20 + 5 = \boxed{37}.$$

Using the same `Value` objects throughout the model allows the scalar engine to differentiate the entire network without a separate neural-network-specific gradient implementation.

5 Experiment: Learning XOR

5.1 Dataset and loss

The notebook defines the following four XOR examples, encoding classes as -1 and $+1$ to match the range of `tanh`:

x_1	x_2	Target y
0	0	-1
0	1	$+1$
1	0	$+1$
1	1	-1

Predictions $\hat{y}_i$ are evaluated with mean squared error:

$$L = \frac{1}{4}\sum_{i=1}^{4}(\hat{y}_i - y_i)^2.$$

5.2 Training procedure

The executed notebook sets random seed 42, learning rate $\eta = 0.05$, and 1000 epochs. Each epoch computes predictions and loss, resets model parameter gradients, invokes `loss.backward()`, and updates every parameter using full-batch gradient descent:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}.$$

The recorded loss is evaluated immediately before that epoch's parameter update; the final predictions are evaluated after all updates.

6 Results and Discussion

Metric	Observed value
Initial recorded MSE (epoch 0)	1.877191
Recorded MSE (epoch 100)	0.127087
Recorded MSE (epoch 500)	0.008296
Final recorded MSE (epoch 999)	0.003368
Training classification accuracy	100%

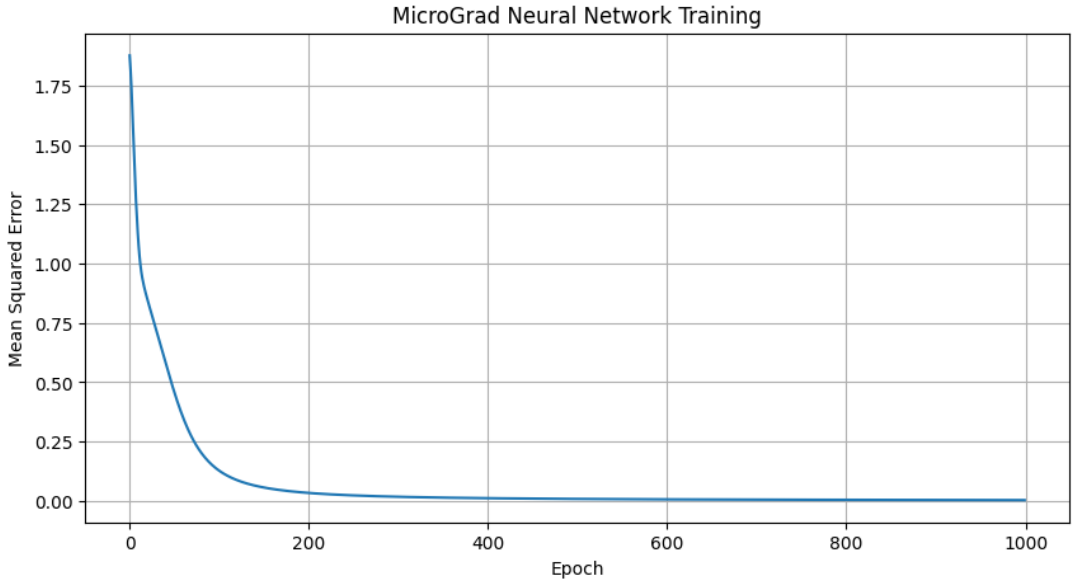


Figure 1: Training loss curve saved in the executed demonstration notebook.

Input	Target	Prediction	Predicted class
$(0, 0)$	-1	-0.9520	-1
$(0, 1)$	$+1$	$+0.9325$	$+1$
$(1, 0)$	$+1$	$+0.9371$	$+1$
$(1, 1)$	-1	-0.9486	-1

The loss fell by approximately 99.82% relative to its first recorded value. The network classified all four training points correctly. This demonstrates successful integration of graph construction, automatic differentiation, and parameter updates on a small nonlinear task. It does not establish performance on unseen data; the reported accuracy is *training* accuracy only.

7 Verification and Limitations

During development, the visible repository tests were executed: nine tests for the scalar engine and three for the neural-network API passed. These results verify the tested behaviors, not every possible edge case or hidden grading criterion. The demonstration uses a tiny manually defined dataset, full-batch updates, and scalar operations, so it is intended for clarity rather than speed or large-scale learning. The current implementation's repeated backward behavior on a reused graph should be treated carefully unless intermediate gradients are reset appropriately.

8 Conclusion

The implementation connects elementary derivative rules to a functioning multilayer neural network. Topological ordering ensures valid gradient propagation, while gradient accumulation handles shared dependencies. The XOR experiment provides a reproducible demonstration of the complete training pipeline, including loss reduction and final predictions.

Reference

Andrej Karpathy, *micrograd* (project inspiration). https://github.com/karpathy/micrograd.
CSE485–CSE488, *Assignment 1: MicroGrad (Scalar Autograd Engine)*, Ain Shams University (assignment specification).